LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack and Queue

Oleh:
Siti Rafeyfa Asyla.P NIM. 2510817220018

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack and Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Siti Rafeyfa Asyla.P
NIM : 2510817220018

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

DAFTAR ISI 3

DAFTAR GAMBAR 4

DAFTAR TABEL 5

SOAL 1 : Implementasi Stack 6

A. Source Code 6

B. Pembahasan 7

SOAL 2 : Si Palui dan Mesin Hitung Aneh 8

A. Source Code 9

B. Output Program 10

C. Pembahasan 10

SOAL 3 : Implementasi Queue 12

A. Source Code 12

B. Pembahasan 14

SOAL 4 : Si Palui dan Mesin Hitung Aneh 15

A. Source Code 16

B. Output Program 17

C. Pembahasan 17

Tautan GIT 19

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1A 10
Gambar 2. Screenshot Hasil Jawaban Soal 1B 10
Gambar 3. Screenshot Hasil Jawaban Soal 1C 10
Gambar 4 Screenshot Hasil Jawaban Soal 2A 17
Gambar 5. Screenshot Hasil Jawaban Soal 2B 17
Gambar 6. Screenshot Hasil Jawaban Soal 2C 17

DAFTAR TABEL

Tabel 1 Source Code Soal 1 6
Tabel 2 Source Code Soal 2 9
Tabel 3 Source Code Soal 3 12
Tabel 4 Source Code Soal 4 16

SOAL 1 : Implementasi Stack

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1 Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = &(s->data[-1]);
}

bool isEmpty(const Stack* s) {
    return (s->top == &(s->data[-1]));
}

bool isFull(const Stack* s) {
    return (s->top == &(s->data[MAX - 1]));
}

void push(Stack* s, int value) {
    if (isFull(s)) {
        throw std::runtime_error("Stack Penuh!");
    }
    s->top++;
    *(s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s)) {
        throw std::runtime_error("Stack Kosong!");
    }
    s->top--;
}
```

```

int peek(const Stack* s) {
    if (isEmpty(s)) {
        throw std::runtime_error("Stack Kosong!");
    }
    return *(s->top);
}
```

B. Pembahasan

Program ini dimulai dengan menyertakan berkas header kustom `"stack.h"` yang mendefinisikan struktur data tumpukan menggunakan array statis dan sebuah pointer `top` sebagai penanda posisi elemen terakhir. Dalam fungsi inisialisasi `init()`, program mengatur pointer top untuk menunjuk ke alamat memori sebelum indeks pertama array (`index -1`) guna menandakan bahwa tumpukan dalam keadaan kosong. Program kemudian mengimplementasikan operasi `push()`, di mana pointer `top` akan bergeser naik (inkremen) ke alamat memori berikutnya sebelum menyimpan nilai baru ke dalam tumpukan, serta operasi `pop()` yang menggeser pointer `top` turun (dekremen) untuk menghapus akses ke elemen teratas sesuai dengan prinsip *Last-In-First-Out* (LIFO).

Fungsi `peek()` digunakan untuk melihat nilai pada posisi yang sedang ditunjuk oleh `top` tanpa mengubah struktur tumpukan tersebut. Kebenaran algoritma ini terjustifikasi melalui penggunaan referensi alamat memori secara langsung yang memastikan manipulasi data terjadi pada posisi yang tepat, didukung oleh proteksi `std::runtime_error` untuk menangani kondisi *overflow* (saat tumpukan penuh) dan *underflow* (saat tumpukan kosong). Berdasarkan analisis Big-O, program ini memiliki kompleksitas waktu O(1) untuk semua fungsi utama karena operasi penambahan, penghapusan, dan pengecekan dilakukan dalam satu langkah konstan melalui manipulasi pointer, serta memiliki kompleksitas ruang O(1) karena penggunaan memori didasarkan pada alokasi array statis dengan ukuran tetap yang tidak bergantung pada jumlah data yang diproses.

SOAL 2 : Si Palui dan Mesin Hitung Aneh

Time Limit: 1.0 s

Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi. Setiap simbol bisa berupa:

• sebuah bilangan bulat, atau

• sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

• Mesin membaca simbol dari kiri ke kanan.

• Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.

• Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

• $1 \leq n \leq 100$

• $-1000 \leq Ai \leq 1000$

Keterangan:

• n adalah jumlah simbol dalam ekspresi

• Ai adalah nilai bilangan pada input

• Simbol yang valid hanya:

‣ bilangan bulat

‣ operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

• Baris pertama n menyatakan jumlah atau panjang simbol

• Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

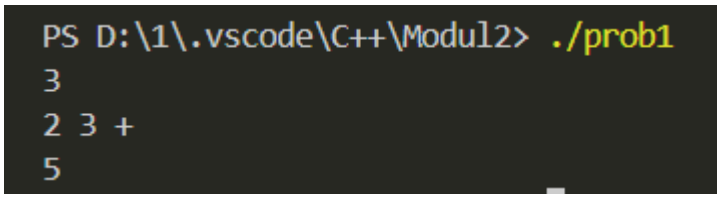
A. Source Code

Tabel 2 Source Code prob1.cpp

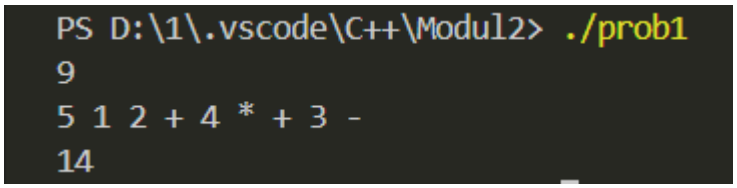
```
#include <iostream>
#include <string>
#include "stack.h"

using namespace std;

int main() {
    int n;
    if (!(cin >> n)) return 0;

    Stack s;
    init(&s);
    string input;

    for (int i = 0; i < n; i++) {
        cin >> input;
        if (input == "+" || input == "-" || input == "*"
|| input == "/") {
```

```
            int bil2 = peek(&s); pop(&s);
            int bil1 = peek(&s); pop(&s);

            if (input == "+") push(&s, bil1 + bil2);
            else if (input == "-") push(&s, bil1 - bil2);
            else if (input == "*") push(&s, bil1 * bil2);
            else if (input == "/") push(&s, bil1 / bil2);
        } else {
            push(&s, stoi(input));
        }
    }
    cout << peek(&s) << endl;
    return 0;
}
```

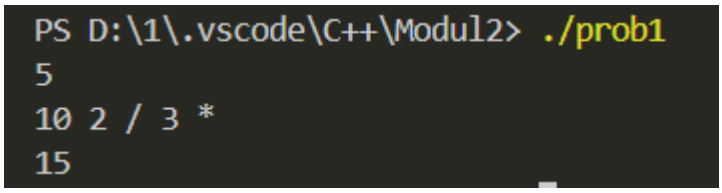
B. Output Program



Gambar 1. Screenshot Hasil Jawaban Soal 2A



Gambar 2. Screenshot Hasil Jawaban Soal 2B



Gambar 3. Screenshot Hasil Jawaban Soal 2C

C. Pembahasan

Program ini dimulai dengan menyertakan pustaka standar `<iostream>` untuk menangani operasi input dan output, serta mengimpor berkas header kustom `"stack.h"` yang mendefinisikan struktur data untuk tumpukan menggunakan array statis dan pointer `top`. Dalam fungsi utama `main()`, program menginstansiasi objek

`s` dari tipe `Stack` dan memanggil fungsi `init()` untuk mengatur posisi awal pointer ke alamat sebelum indeks pertama array. Program kemudian menjalankan struktur perulangan untuk menerima input pengguna berupa sejumlah simbol yang merepresentasikan angka atau operator matematika dalam format postfix.

Setelah data diterima, kode melakukan evaluasi ekspresi dengan cara memasukkan angka ke dalam stack menggunakan fungsi `push()`, dan jika menemukan operator, program memanggil fungsi `peek()` serta `pop()` untuk mengambil dua nilai teratas guna dilakukan kalkulasi aritmatika. Kebenaran algoritma ini terjustifikasi melalui prinsip *Last-In-First-Out* (LIFO) yang secara akurat memproses urutan operasi tanpa membutuhkan tanda kurung, didukung oleh proteksi `runtime_error` untuk menangani kondisi stack penuh atau kosong. Berdasarkan analisis Big-O, program ini memiliki kompleksitas waktu O(1) untuk setiap operasi stack karena akses langsung melalui pointer tanpa perulangan, serta memiliki kompleksitas ruang O(1) karena penggunaan memori hanya dialokasikan untuk array statis dengan ukuran tetap yang tidak bergantung pada besar input.

SOAL 3 : Implementasi Queue

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw. Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 3 Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>
#include <iostream>

void init(Queue* q) {
    q->front = NULL;
    q->rear = NULL;
}

bool isEmpty(const Queue* q) {
    return (q->front == NULL);
}

bool isFull(const Queue* q) {
    if (isEmpty(q)) return false;

    int* nextRear;
    if (q->rear == &(q->data[MAX - 1])) {
        nextRear = (int*)&(q->data[0]);
    } else {
        nextRear = q->rear + 1;
    }
    return (nextRear == q->front);
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) {
        throw std::runtime_error("Queue Penuh!");
    }

```

```
    if (isEmpty(q)) {
        q->front = &(q->data[0]);
        q->rear = &(q->data[0]);
    } else {
        if (q->rear == &(q->data[MAX - 1])) {
            q->rear = &(q->data[0]);
        } else {
            q->rear++;
        }
    }
    *(q->rear) = value;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) {
        throw std::runtime_error("Queue Kosong!");
    }

    if (q->front == q->rear) {
        q->front = NULL;
        q->rear = NULL;
    } else {
        if (q->front == &(q->data[MAX - 1])) {
            q->front = &(q->data[0]);
        } else {
            q->front++;
        }
    }
}

int front(const Queue* q) {
    if (isEmpty(q)) throw std::runtime_error("Queue Kosong!");
    return *(q->front);
}

int back(const Queue* q) {
    if (isEmpty(q)) throw std::runtime_error("Queue Kosong!");
    return *(q->rear);
}
```

B. Pembahasan

Program ini dimulai dengan menyertakan berkas header kustom "`stack.h`" yang mendefinisikan struktur data tumpukan menggunakan array statis dan sebuah pointer `top` sebagai penanda posisi elemen terakhir. Dalam fungsi inisialisasi `init()`, program mengatur pointer `top` untuk menunjuk ke alamat memori sebelum indeks pertama array (`indeks -1`) guna menandakan bahwa tumpukan dalam keadaan kosong. Program kemudian mengimplementasikan operasi `push()`, di mana pointer `top` akan bergeser naik (inkremen) ke alamat memori berikutnya sebelum menyimpan nilai baru ke dalam tumpukan, serta operasi `pop()` yang menggeser pointer `top` turun (dekremen) untuk menghapus akses ke elemen teratas sesuai dengan prinsip *Last-In-First-Out* (LIFO).

Fungsi `peek()` digunakan untuk melihat nilai pada posisi yang sedang ditunjuk oleh top tanpa mengubah struktur tumpukan tersebut. Kebenaran algoritma ini terjustifikasi melalui penggunaan referensi alamat memori secara langsung yang memastikan manipulasi data terjadi pada posisi yang tepat, didukung oleh proteksi `std::runtime_error` untuk menangani kondisi *overflow* saat tumpukan penuh dan *underflow* saat tumpukan kosong. Berdasarkan analisis Big-O, program ini memiliki kompleksitas waktu O(1) untuk semua fungsi utama karena operasi penambahan, penghapusan, dan pengecekan dilakukan dalam satu langkah konstan melalui manipulasi pointer, serta memiliki kompleksitas ruang O(1) karena penggunaan memori didasarkan pada alokasi array statis dengan ukuran tetap yang tidak bergantung pada jumlah data yang diproses.

SOAL 4 : Si Palui dan Mesin Hitung Aneh

Time Limit: 1.0 s

Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

• $1 \leq n \leq 100$

• $-1 \leq k \leq n$

• $-1000 \leq Ai \leq 1000$

Keterangan:

• n: jumlah elemen

• k: ukuran jendela

• Ai : elemen array

Format Input

n

k A1 A2 ... An

Keterangan:

• Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk

• Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

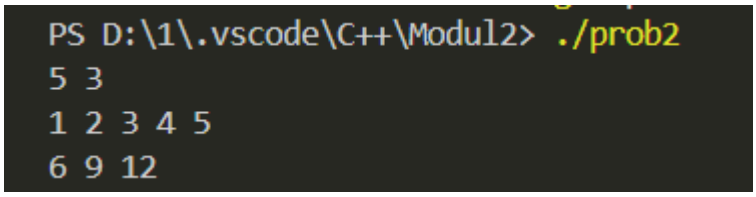
A. Source Code

Tabel 4. Source Code prob2.cpp

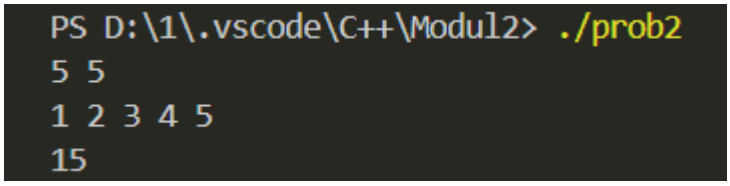
```
#include <iostream>
#include "queue.h"

using namespace std;

int main() {
    int n, k;
    if (!(cin >> n >> k)) return 0;

    int angka[100];
    for (int i = 0; i < n; i++) {
        cin >> angka[i];
    }

    for (int i = 0; i <= n - k; i++) {
        Queue q;
        init(&q);
        int total = 0;

        for (int j = i; j < i + k; j++) {
            enqueue(&q, angka[j]);
        }

        while (!isEmpty(&q)) {
            total += front(&q);
            dequeue(&q);
        }

        cout << total;
        if (i < n - k) cout << " ";
    }

```

```
    cout << endl;
    return 0;
}
```

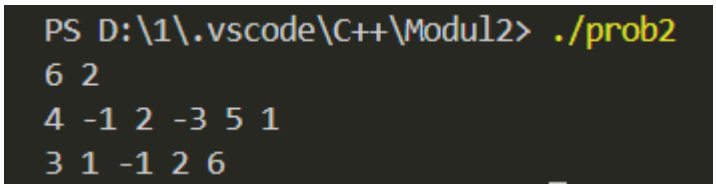
B. Output Program



Gambar 4. Screenshot Hasil Jawaban Soal 4A



Gambar 5. Screenshot Hasil Jawaban Soal 4B



Gambar 6. Screenshot Hasil Jawaban Soal 4C

C. Pembahasan

Program ini dimulai dengan menyertakan pustaka standar `<iostream>` untuk menangani operasi input dan output, serta mengimpor berkas header kustom `"queue.h"` yang mendefinisikan struktur data antrean menggunakan pointer `front` dan `rear`. Dalam fungsi utama `main()`, program menerima input pengguna berupa jumlah elemen total `n` dan ukuran jendela `k`, kemudian menyimpan deretan angka ke dalam array statis. Program kemudian menjalankan struktur perulangan *nested* untuk mensimulasikan mekanisme *sliding window*, di mana pada setiap pergeseran, objek `q` dari tipe `Queue` diinstansiasi dan diinisialisasi ulang menggunakan fungsi `init()`.

Setelah data jendela ditentukan, kode memanggil fungsi `enqueue()` untuk memasukkan elemen-elemen dalam rentang jendela tersebut ke dalam antrean, lalu

menjumlahkannya dengan cara memanggil fungsi `front()` dan `dequeue()` secara berulang hingga antrean kosong. Kebenaran algoritma ini terjustifikasi melalui prinsip *First-In-First-Out* (FIFO) yang secara sistematis memproses sub-kelompok angka secara berurutan, didukung oleh logika pembatas `n - k` yang memastikan jendela tidak melewati batas indeks array. Berdasarkan analisis Big-O, program ini memiliki kompleksitas waktu O(n . k) karena adanya perulangan di dalam perulangan untuk memproses setiap jendela, serta memiliki kompleksitas ruang O(1) di luar alokasi array utama, karena memori untuk objek `Queue` hanya dialokasikan dalam jumlah tetap sesuai kapasitas maksimal yang didefinisikan.

Tautan GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-srafeyfaa71.git